



دانشکده سامانه‌های هوشمند

بینایی ماشین

استاد درس: دکتر حائیه نادری

هندسه تحلیلی : خط، تقاطع دو خط و شباهت برداری

IMPORTS

کتابخانه های NUMPY , MATPLOTLIB در این تمرین استفاده شدند.

LINE EQUATION

تابع Line from two points

این تابع یک خط راست را از روی دو نقطه در صفحه به دست می آورد و معادله آن را به فرم زیر برمی گرداند.

$$ax + by + c = 0$$

این تابع در واقع کاری می کند که اگر دو نقطه روی یک صفحه داشته باشیم، بتوانیم معادله خطی را داشته باشیم که از هر دو عبور می کند.

```
def line_from_points(p1, p2):
```

تابع دو نقطه به عنوان ورودی می گیرد: $p1 = (x1, y1)$ $p2 = (x2, y2)$

```
x1, y1 = p1  
x2, y2 = p2
```

مختصات هر نقطه از tuple استخراج می شود.

```
a = y1 - y2  
b = x2 - x1  
c = x1 * y2 - x2 * y1
```

ضرایب معادله خط : این ضرایب طوری انتخاب می شوند که هر دو نقطه معادله را ارضا کند.

$$c = \begin{vmatrix} x_1 & y_1 \\ x_2 & y_2 \end{vmatrix}$$

a: شیب عمودی b: شیب افقی c: بخش جهت دار (از دترمینان می آید)

```
return a, b, c
```

تابع یک سه تایی برمی گرداند که معادله خط را مشخص می کند.

تابع print_line

```
def print_line(L, name="Line"):
```

ورودی ها:

- L: یک سه تایی مثل (a, b, c) که ضرایب معادله خط را نگه می دارد .
- Name: یک نام اختیاری برای خط (پیش فرض آن "Line" است)

```
a, b, c = L  
print(f"{name}: {a}x + {b}y + {c} = 0")
```

این خط مقدارهای داخل L را جدا می کند : a: ضریب x b: ضریب y c: عدد ثابت

در نهایت خط آخر کد، معادله خط را چاپ می کند.

تابع Intersection & classification

هدف این تابع این است که دو خط را بررسی کند ببیند موازی اند، یکسان اند یا همدیگر را قطع می کنند و اگر قطع شوند، نقطه ی تقاطع را حساب کند.

```
def classify_and_intersect(L1, L2):
```

ورودی ها : L1: ضرایب خط اول (a1, b1, c1) و L2: ضرایب خط دوم (a2, b2, c2)

```
a1, b1, c1 = L1
a2, b2, c2 = L2
```

اینجا ضرایب هر خط از tuple جدا می شود

```
D = a1 * b2 - a2 * b1
```

این خط مقدار دترمینان (Determinant) را حساب می کند. تفسیر :

- اگر $D = 0$ -- دو خط موازی یا یکسان
- اگر $D \neq 0$ -- دو خط همدیگر را قطع می کنند (چون شیب شان برابر نیست)

```
if np.isclose(D, 0):
```

این خط بررسی می کند آیا D تقریباً صفر است یا نه.

- np.isclose برای جلوگیری از خطای اعشاری استفاده می شود
- چون ممکن است عدد دقیقاً ۰ نشود .

```
if np.isclose(a1 * c2, a2 * c1) and np.isclose(b1 * c2, b2 * c1):
```

این شرط بررسی می کند آیا دو خط کاملاً یکسان هستند یا نه. ایده : اگر نسبت ضرایب هر سه بخش برابر باشد، خطوط یکی هستند

در واقع عبارت روبه رو بررسی میشود:

$$\frac{a_1}{a_2} = \frac{b_1}{b_2} = \frac{c_1}{c_2}$$

```
return "identical", None
```

اگر خطوط یکی باشند، خروجی :

- "identical"
- و هیچ نقطه تقاطعی نداریم یعنی none

```
else:
    return "parallel", None
```

اگر فقط موازی باشند (نه یکسان)، خروجی :

- "parallel"
- بدون نقطه تقاطع

$$x = (b1 * c2 - b2 * c1) / D$$
$$y = (c1 * a2 - c2 * a1) / D$$

- حالت دوم وقتی است که خطوط قطع می‌شوند. (دترمینان ≠ نیست)
- محاسبه مختصات x, y نقطه تقاطع با فرمول کرامر
- صورت های کسر دترمینان x و y هستند. تقسیم بر D جواب واقعی را نرمال می‌کند.

```
return "intersecting", (x, y)
```

اگر خطوط همدیگر را قطع کنند:

- نوع "intersecting"
- نقطه تقاطع (x, y)

تابع Vector Analysis

این تابع می‌گوید: این دو بردار چقدر به هم شبیه‌اند و چه زاویه‌ای دارند. این تابع دو کار را برای بردارهای $2d$ و $3d$ انجام می‌دهد:

۱. محاسبه ضرب داخلی (dot product)
۲. محاسبه شباهت کسینوسی (cosine similarity)

```
def vector_analysis(v1, v2):
```

این تابع دو تا بردار به عنوان ورودی می‌گیرد.

```
v1 = np.array(v1)
v2 = np.array(v2)
```

اینجا بردارها تبدیل به آرایه NumPy می‌شوند. زیرا نامپای محاسبات برداری را سریع و ساده می‌کند و تابع‌های آماده مثل `dot` و `norm` دارد.

```
dot = np.dot(v1, v2)
```

ضرب داخلی (Dot Product) دو بردار محاسبه می‌شود.

$$(1, 2) \cdot (3, 4) = 1 \times 3 + 2 \times 4 = 11$$

روش دیگر به دست آوردن ضرب داخلی:

$$v_1 \cdot v_2 = \|v_1\| \|v_2\| \cos(\theta)$$

به طور خلاصه: ضرب داخلی به ما می گوید که دو بردار چقدر با هم هم راستا هستند. هرچه زاویه کمتر باشد، مقدار کسینوس به ۱ نزدیک تر و ضرب داخلی بزرگتر می شود.

```
norm1 = np.linalg.norm(v1)
norm2 = np.linalg.norm(v2)
```

magnitude/norm بردار v1 و v2 را به دست می آوریم.

$$||v|| = \sqrt{x^2 + y^2}$$

```
if np.isclose(norm1, 0) or np.isclose(norm2, 0):
    cos_sim = 0
```

جهت جلوگیری از تقسیم بر صفر، اگر یکی از بردارها صفر باشد:

- طولش = ۰
- فرمول کسینوس خراب می شود

پس شباهت را ۰ فرض می کنیم

```
else:
    cos_sim = dot / (norm1 * norm2)
```

$$\cos(\theta) = \frac{v1 \cdot v2}{||v1|| ||v2||}$$

شباهت کسینوسی را به دست می آوریم.

در نهایت

- اگر $\cos = 1$ پس دو بردار هم جهت هستند.
- اگر $\cos = 0$ ، دو بردار برهم عمود هستند.
- اگر $\cos = -1$ ، دو بردار خلاف جهت هستند.

```
return dot, cos_sim
```

خروجی:

- مقدار ضرب داخلی
- مقدار شباهت کسینوسی

این تابع کارش این است که عدد cosine similarity را به یک جمله‌ی قابل فهم تبدیل کند.

```
def interpret_cosine(cos):
```

ورودی این تابع COS یا همان مقدار شباهت کسینوسی (بین -۱ و ۱)

```
if np.isclose(cos, 1):
    return "Same direction"
```

اگر مقدار COS تقریباً برابر ۱ باشد، خروجی "Same direction" است.

- یعنی زاویه بین دو بردار 0°
- بردارها هم جهت هستند

```
elif np.isclose(cos, -1):
    return "Opposite direction"
```

اگر cosine تقریباً -۱ باشد، خروجی "Opposite direction" است.

- زاویه 180°
- بردارها دقیقاً خلاف جهت هم هستند

```
elif np.isclose(cos, 0):
    return "Orthogonal (90°)"
```

اگر cosine تقریباً ۰ باشد، خروجی "Orthogonal (90°)" است.

- زاویه 90° درجه
- بردارها عمود هستند

```
elif cos > 0:
    return "Acute angle"
```

اگر cosine مثبت باشد (ولی ۱ نباشد)، خروجی "Acute angle" است.

- زاویه بین ۰ تا 90° درجه
- یعنی بردارها تا حدی هم جهت‌اند

```
else:
    return "Obtuse angle"
```

اگر هیچ کدام از بالا نبود، خروجی "Obtuse angle" است.

- یعنی cosine منفی است (بین ۰ و -۱)
- زاویه بین ۹۰ تا ۱۸۰ درجه

Plot Functions

تابع Plot Lines

این تابع وظیفه رسم دو خط در صفحه دوبعدی و مشخص کردن محل برخورد آنها را بر عهده دارد.

تابع Plot vectors

این تابع دو بردار را بسته به ابعادشان (2D/ 3D) در یک نمودار بصری ترسیم می کند.

Run Functions

تابع analyze_and_plot_lines

این تابع به عنوان یک واحد مدیریت کننده (Wrapper)، تمام مراحل تحلیل دو خط را به صورت یکپارچه انجام می دهد.

۱. محاسبه معادلات: ابتدا با دریافت چهار نقطه، معادلات خطوط را به فرم استاندارد ($ax+by+c=0$) محاسبه و در خروجی چاپ می کند.

۲. بررسی وضعیت: رابطه هندسی بین دو خط (مقاطع، موازی یا منطبق) را تعیین کرده و در صورت وجود برخورد، مختصات دقیق نقطه تلاقی را نمایش می دهد.

۳. خروجی بصری: در نهایت، با فراخوانی تابع رسم، نمودار خطوط و محل برخورد آنها را برای درک بهتر هندسی تولید می کند.

تابع analyze_and_plot_vectors

این تابع فرآیند تحلیل عددی و هندسی دو بردار را مدیریت کرده و نتایج را به صورت متنی و تصویری ارائه می دهد:

۱. تحلیل عددی: مقدار ضرب داخلی (Dot Product) و شباهت کسینوسی (Cosine Similarity) را محاسبه کرده و اطلاعات اولیه بردارها را در کنسول چاپ می کند.

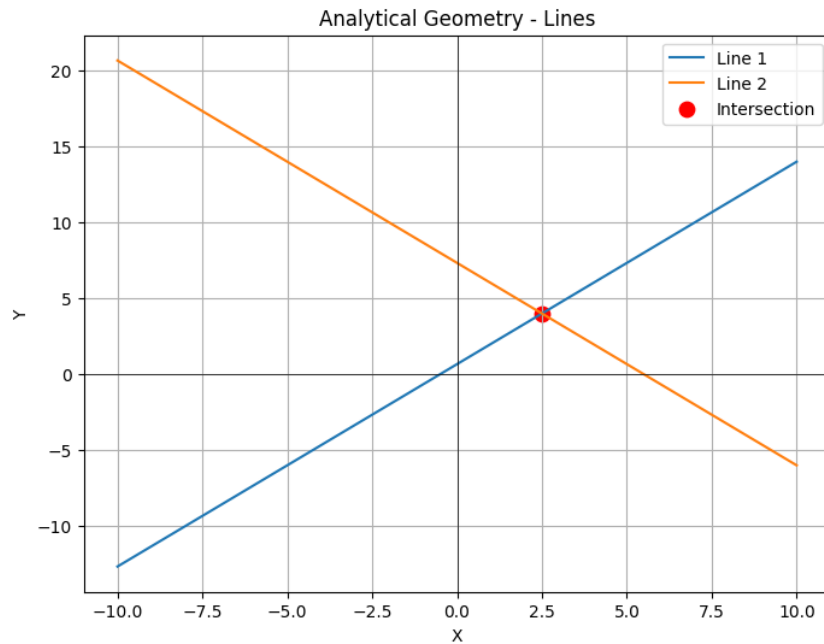
۲. تفسیر هندسی: با استفاده از مقدار کسینوس، وضعیت قرارگیری بردارها نسبت به هم (مانند هم جهت بودن یا عمود بودن) را به زبان ساده تفسیر می کند.

۳. نمایش بصری: بر اساس ابعاد ورودی، نمودار دوبعدی یا سه بعدی متناظر را رسم می کند تا زاویه و جهت بردارها به صورت گرافیکی قابل مشاهده باشد.

```

--- [ Lines Analysis ] ---
Line 1:  $-4x + 3y + -2 = 0$ 
Line 2:  $4x + 3y + -22 = 0$ 
Relation: intersecting
Intersection Point: (2.50, 4.00)

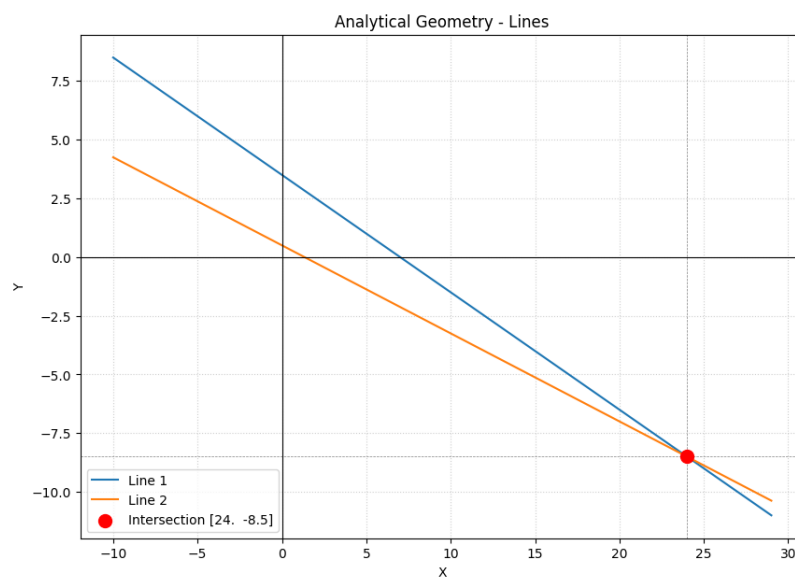
```



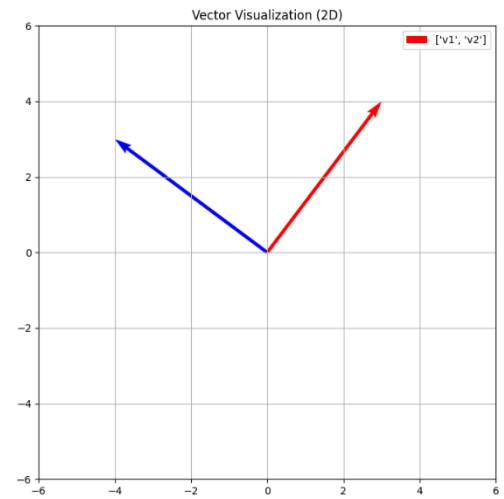
```

--- [ Lines Analysis ] ---
Line 1:  $-4x + -8y + 28 = 0$ 
Line 2:  $-3x + -8y + 4 = 0$ 
Relation: intersecting
Intersection Point: (24.00, -8.50)

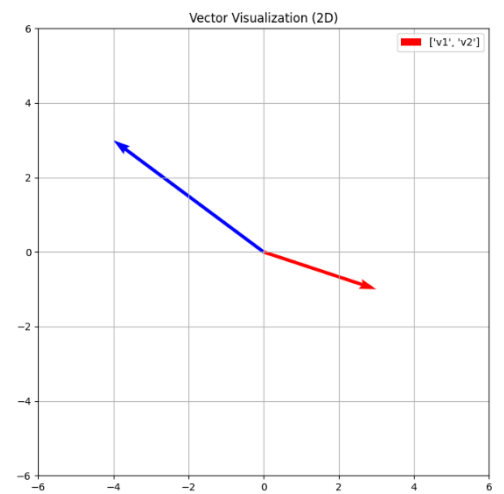
```



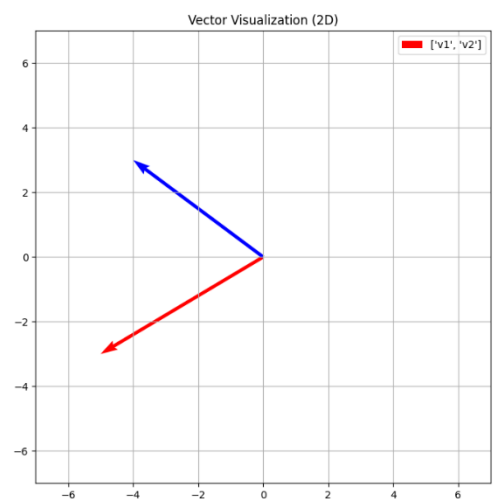
--- [2D Vector Analysis] ---
Vector 1: [3, 4] | Vector 2: [-4, 3]
Dot Product: 0
Cosine Similarity: 0.0000
Interpretation: Orthogonal (90°)



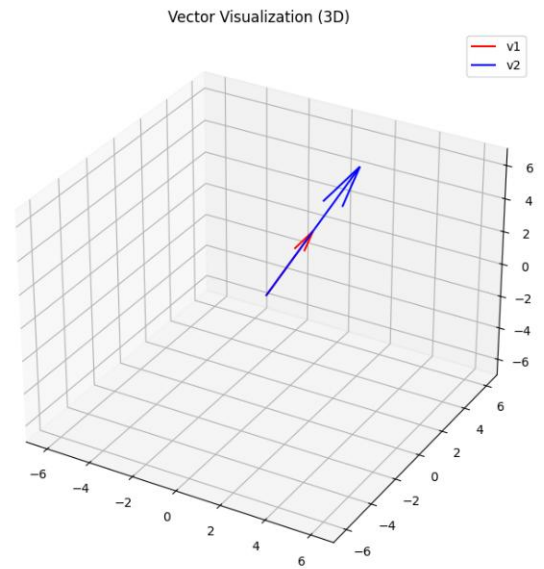
--- [2D Vector Analysis] ---
Vector 1: [3, -1] | Vector 2: [-4, 3]
Dot Product: -15
Cosine Similarity: -0.9487
Interpretation: Obtuse angle



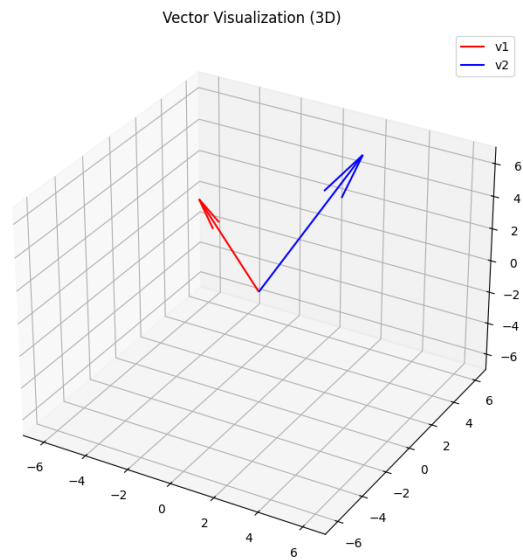
--- [2D Vector Analysis] ---
Vector 1: [-5, -3] | Vector 2: [-4, 3]
Dot Product: 11
Cosine Similarity: 0.3773
Interpretation: Acute angle



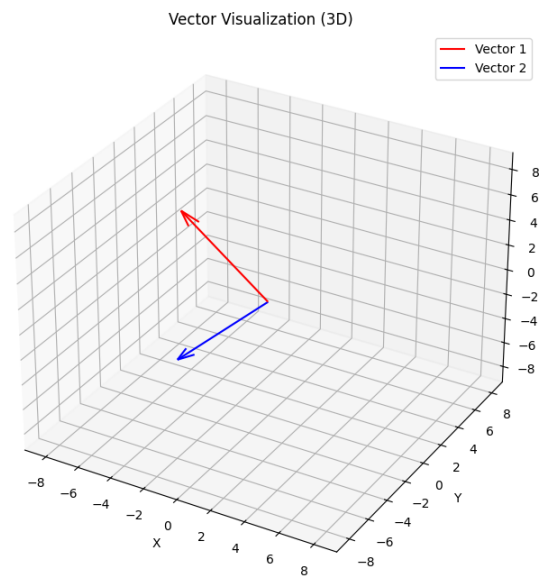
--- [3D Vector Analysis] ---
Vector 1: [1, 2, 3] | Vector 2: [2, 4, 6]
Dot Product: 28
Cosine Similarity: 1.0000
Interpretation: Same direction



--- [3D Vector Analysis] ---
Vector 1: [-4, 2, 3] | Vector 2: [2, 5, 6]
Dot Product: 20
Cosine Similarity: 0.4607
Interpretation: Acute angle



--- [3D Vector Analysis] ---
Vector 1: [-4, -2, 7] | Vector 2: [-5, -1, -6]
Dot Product: -20
Cosine Similarity: -0.3058
Interpretation: Obtuse angle



نتیجه گیری:

کد پیاده سازی شده با بهره گیری از جبر خطی، توانایی تحلیل دقیق روابط هندسی بین خطوط و بردارها را داراست. در بخش خطوط، با استفاده از فرم عمومی معادله خط $ax+by+c=0$ ، تشکیل دستگاه معادلات، وضعیت دو خط (مقاطع، موازی یا منطبق) به دقت تعیین و نقطه برخورد با محاسبات ماتریسی استخراج می گردد.

در بخش بردارها، نتایج نشان می دهد که ضرب داخلی و شباهت کسینوسی فراتر از یک محاسبه عددی ساده، معیاری هوشمند برای درک جهت گیری فضایی هستند؛ به طوری که هر چه شباهت کسینوسی به عدد یک نزدیک تر باشد، ضرب داخلی بزرگ تر میشود و بردارها هم راستا تر هستند. و هر چه شباهت کسینوسی به صفر میل کند، ضرب داخلی کوچک تر شده و نشان دهنده عمود بودن (تعامد) و استقلال جهت آنهاست.